INF-253 Lenguajes de Programación
Tarea 2 2026-1: Rey Destronado

Profesores: José Luis Martí Lara, Wladimir Ormazábal Orellana
Ayudante coordinador: Bastián Ríos Lastarria

30 de marzo de 2026

Índice

1. Contexto **2**
1.1. Objetivo Principal 2

2. Descripción de la tarea **2**
2.1. Especificaciones generales y progresión 2
2.2. Flujo de Turnos 2
2.3. El Arsenal del Rey 3
2.4. Piezas enemigas y comportamiento 3
2.5. Aparición de las piezas 4

3. Archivos y Módulos **4**
3.1. Estructura general y restricciones 4
3.2. Módulos y funciones/estructuras obligatorias 5
3.2.1. main.c/.h 5
3.2.2. tablero.c/.h 5
3.2.3. piezas.c/.h 5
3.2.4. armas.c/.h (punteros a función obligatorios) 5

4. Interfaz Visual (HUD) **6**

5. Sobre la entrega **6**

6. Calificación **8**
6.1. Entrega 8
6.1.1. Entrega Mínima (total 30 pts) 8
6.1.2. Asignación de puntajes general (total 70 pts) 8
6.2. Descuentos 9

1. Contexto

Durante años, el Rey ha sido un tirano despiadado con sus súbditos. Cansadas de sus malos tratos, las piezas de su propio reino han organizado un gran alzamiento para destronarlo. Acorralado, sin guardias y con el castillo tomado, el Rey logra atrincherarse en la antigua bóveda de armas prohibidas. Tu objetivo es volver a tomar control sobre tu castillo y sobrevivir a la horda de la revolución.
Tu misión es desarrollar un videojuego por turnos en consola escrito en C, donde el jugador controlará a este Rey armado hasta los dientes. Deberás gestionar la memoria dinámica de un tablero bidimensional que cambiará de tamaño, implementar el comportamiento de las piezas enemigas y crear un sistema modular de armamento utilizando punteros a función.

1.1. Objetivo Principal

Implementar un juego de supervivencia en **tablero bidimensional administrado mediante memoria dinámica** (*triple puntero* `void ***`). El estudiante deberá modelar distintas piezas de ajedrez con reglas de movimiento específicas y desarrollar un inventario de acciones (armas) empleando **obligatoriamente** arreglos de punteros a función.

2. Descripción de la tarea

El juego se desarrolla en un tablero de tamaño variable y toda la interacción se realiza por terminal mediante un sistema de turnos.

2.1. Especificaciones generales y progresión

- **Tableros dinámicos** (Niveles): El juego consta de 3 niveles consecutivos. Al limpiar un nivel de enemigos, se avanza al siguiente, regenerando el tablero con un nuevo tamaño (usando memoria dinámica):
  1. **Nivel 1** (Plaza principal): **Tablero de 12x12**.
  2. **Nivel 2** (Jardín del Rey): **Tablero de 8x8**.
  3. **Nivel 3** (Entrada del castillo - Ubicación Final): **Tablero de 6x6**.
- **Condición de Victoria**: El jugador avanza de nivel al eliminar a todas las piezas enemigas del tablero actual. El juego se gana al limpiar el Nivel 3.
- **Condiciones de Derrota Crítica**: El jugador pierde inmediatamente si una pieza enemiga se mueve a la casilla ocupada por el Rey en ese momento.

2.2. Flujo de Turnos

El juego es estrictamente por turnos y se controla mediante el teclado y por terminal:

1. Turno del Rey (Jugador): El jugador ingresa una tecla para actuar:
   - **Movimiento (Q, W, E, A, S, D, Z, C):** El Rey se mueve 1 casilla. **W** (Arriba), **S** (Abajo), **A** (Izquierda), **D** (Derecha), **Q** (Arriba-Izq), **E** (Arriba-Der), **Z** (Abajo-Izq), **C** (Abajo-Der).

- Nota: Moverse es la única forma de recargar munición, y **sólo recarga la Escopeta** (con un máximo de 2 balas).
- **Disparar (1, 2, 3, 4)**: El jugador presiona el número correspondiente al arma. Acto seguido, el sistema solicita la dirección del disparo usando las mismas teclas de movimiento **(ej: presionar 1 y luego W dispara la escopeta hacia arriba).**
- Toda acción (moverse o disparar) **consume** el turno. Acciones inválidas no consumen turno.

2. **Turno de las Piezas (Enemigos):** Tras la acción del Rey, todas las piezas enemigas vivas se mueven hacia el Rey respetando sus reglas de movimientos típicas del juego de ajedrez *(con algunas limitaciones).* En caso de que el Rey se encuentre en una casilla alcanzable por una pieza, **el Rey muere y el jugador pierde.**

2.3. El Arsenal del Rey

Las armas operan mediante un arreglo de punteros a función. Aclaración de munición: Avanzar de un Nivel a otro recarga la munición de todas las armas al máximo. Durante el transcurso de un nivel, solo la Escopeta puede recuperar munición (moviéndose).

1. **Escopeta (Shotgun)**:
   Munición Máx: 2 (Recupera su munición por cada turno de movimiento).
   Efecto: Inflige 2 de daño a la primera pieza a distancia 1 en la dirección elegida, y 1 de daño a las piezas ubicadas en las tres casillas inmediatamente detrás de ella.

2. **Rifle de Francotirador (Sniper)**:
   Munición Máx: 1
   Efecto: Alcance infinito en una línea recta. Inflige 3 de daño a la primera pieza impactada. No atraviesa.

3. **Granada Real (AoE)**:
   Munición Máx: 2
   Efecto: El jugador ingresa la dirección y se lanza a exactamente 3 casillas de distancia. Genera una explosión en un área de 3x3 centrada en el impacto, infligiendo 2 de daño a todo lo que esté en su radio. Se pueden eliminar los lanzamientos diagonales si así lo desean, en caso de incluirlos, pueden ajustar la distancia de lanzamiento a su conveniencia.

4. **Arma especial (Definida por el estudiante):**
   La munición, daño y efecto debe ser **definida por el estudiante**, no necesariamente debe ser un arma, podría ser un potenciador, un efecto de estado al Rey o piezas enemigas, etc. La idea es que sea algo balanceado y creativo, hay completa libertad siempre y cuando no interfiera con la experiencia de juego general, y debe estar documentada correctamente lo que realiza, y **cómo** se realiza, es decir, explicar el código detrás de la implementación de la función y arma. ¡Sean creativos e innovadores!

2.4. Piezas enemigas y comportamiento

Cada pieza negra busca acercarse al Rey para capturarlo. Considerar que siempre buscarán la ruta más corta para lograr este objetivo, solamente puede haber una pieza por celda y tendrán que manejar estas colisiones de manera correcta.

1. **Peón:** $HP = 1$. Avanza 1 casilla (ortogonal hacia el Rey). Si el Rey está en su diagonal frontal a 1 casilla, ataca en diagonal, **también puede atacar frontal al Rey.**
2. **Caballo:** $HP = 2$. Se mueve en forma de 'L'. Tratará de elegir el salto que minimice la distancia hacia el Rey. Puede saltar sobre otras piezas. (se recomienda buscar imágenes referenciales para entender claramente el movimiento del caballo)
3. **Alfil:** $HP = 2$. Se mueve únicamente en diagonal (hasta 3 casillas por turno).
4. **Torre:** $HP = 4$. Se mueve únicamente en líneas ortogonales (arriba, abajo, izquierda, derecha) hasta 3 casillas por turno. Para balancear su dureza, la Torre es pesada y **solo se mueve una vez cada 2 turnos** de los enemigos.
5. **La Reina:** $HP = 3$. **Solo aparece en el Nivel 3**. Combina los movimientos de la Torre y el Alfil (ortogonal y diagonal) moviéndose hasta 4 casillas por turno para acechar implacablemente al Rey.

2.5. Aparición de las piezas

El Rey siempre aparece en una posición aleatoria dentro la última línea del tablero sin contar las esquinas.
Las piezas enemigas siempre aparecerán también en posiciones aleatorias al lado extremo opuesto del Rey (primera fila del tablero) **a excepción** de los peones, quienes aparecerán también en posiciones aleatorias en la **segunda** fila.

Niveles y cantidad de enemigos

- **Nivel 1:** 4 peones, 2 caballos y 2 alfiles.
- **Nivel 2:** 4 peones, 2 caballos y 2 torres.
- **Nivel 3:** 2 peones, 1 reina, 1 alfil y 1 torre.

3. Archivos y Módulos

Esta sección fija la organización mínima en archivos y los tipos de datos esenciales. **Se exige el uso de compilación separada** mediante `Makefile`. El uso de la memoria dinámica es fundamental, ya que el tablero debe ser destruido y recreado con nuevas dimensiones al cambiar de nivel.

3.1. Estructura general y restricciones

- Tablero con triple puntero: Es obligatorio representar la grilla como `void ***`.
- Manejo de Memoria: Al pasar del Nivel 1 (12x12) al Nivel 2 (8x8) y al Nivel 3 (6x6), el programa debe liberar completamente la memoria del tablero anterior (*free*) antes de alojar el nuevo. Existe descuento por *memory leaks*.
- Compilación: Declarar prototipos y structs en archivos **.h**, definir la lógica en **.c**

3.2. Módulos y funciones/estructuras obligatorias

3.2.1. main.c/.h

```
typedef struct {
    Tablero *t;
    Armas arsenal;
    Pieza *jugador; /* El Rey */
    int nivel_actual; /* 1, 2 o 3 */
    int turno_enemigos; /* Contador para gestionar piezas lentas como la Torre */
} Juego;
// Rol: Inicia el ciclo de juego, maneja el bucle principal de entrada (WASD+QEZC / 1234),
gestiona el avance de niveles y libera la memoria general al terminar o perder.
```

3.2.2. tablero.c/.h

```
typedef struct {
    int W, H;
    void ***celdas; /* celdas[y][x] apunta a Celda* */
} Tablero;
/* Es completamente imperativo que el Tablero se maneje de la manera entregada */

struct Tablero* tablero_crear(int ancho, int alto);
void tablero_imprimir(struct Juego *juego);
void tablero_liberar(struct Tablero *tablero); /* debe limpiar la memoria heap */
```

3.2.3. piezas.c/.h

```
typedef struct {
    char tipo; /* 'P'=Peon, 'C'=Caballo, 'A'=Alfil, 'T'=Torre, 'Q'=Reina, 'R'=Rey */
    int hp;
    int x, y;
} Pieza;

typedef struct {
    Pieza *pieza; /* NULL si la celda esta vacia */
} Celda;

void spawn_nivel(struct Juego *juego, int nivel);
void mover_enemigos(struct Juego *juego);
bool verificar_estado_rey(struct Juego *juego); /* Revisa si el Rey esta en Jaque */
```

3.2.4. armas.c/.h (punteros a función obligatorios)

```
struct Juego; /* Forward declaration */
typedef bool (*FuncArma)(struct Juego *j, int dir_x, int dir_y); /* Puntero a funcion */

typedef struct {
    int municion_actual[4]; /* Se puede cambiar */
    int municion_maxima[4]; /* Se puede cambiar */
    FuncArma disparar[4]; /* Arreglo de punteros a funcion con las 4 armas */
} Armas;

bool escopeta(struct Juego *j, int dir_x, int dir_y);
bool francotirador(struct Juego *j, int dir_x, int dir_y); /* Ej (x,y) = (1,0), (0,-1)... */
bool granada(struct Juego *j, int target_x, int target_y);
bool especial(struct Juego *j, int dir_x, int dir_y); /* Se puede modificar */
```

```
/* Es imperativo el uso del arreglo FuncArma para utilizar las funciones de disparo */
/* Ej de uso en main : juego->arsenal.disparar[3](juego, input_x, input_y); */
```

4. Interfaz Visual (HUD)

El programa debe renderizar el tablero en turno, mostrando claramente el estado del Rey, la munición, y advirtiendo en qué nivel se encuentra. A continuación se presenta un ejemplo de lo que se espera visualmente.

```
NIVEL 2 COMPLETADO!
El Rey entra en el castillo...
Recuperando municion total.
==================================================
Nivel: 3 (Entrada del castillo) | Enemigos restantes: 5
Arsenal: [1] Escopeta (2/2) | [2] Sniper (1/1)
         [3] Granada (1/1)  | [4] {arma_especial} (3/3)
=========================

6 [ ][ ][Q][A][ ][T]
5 [ ][P][ ][ ][P][ ]
4 [ ][ ][ ][ ][ ][ ]
3 [ ][ ][ ][ ][ ][ ]
2 [ ][ ][ ][ ][ ][ ]
1 [ ][ ][R][ ][ ][ ]
   1  2  3  4  5  6
ACCIONES: Disparo: [1] Escopeta [2] Sniper [3] Granada [4] Especial
Movimiento:
[Q][W][E]
[A] R [D]
[Z][X][C]

> Ingrese accion (Tecla de Movimiento o Numero de Arma): 2
> Direccion de disparo (Q, W, E, A, S, D, Z, C): W
IMPACTO!! - 3 HP inflingido a La Reina.
La Reina ha caido!! (HP = 0/3)
Las piezas avanzan...
```

Se subirá un video/archivo por separado con un ejemplo más fluido más adelante! Work in Progress...

5. Sobre la entrega

- El código debe venir indentado y ordenado.
- Se debe entregar los siguientes archivos:
  - main.c & main.h
  - tablero.c & tablero.h
  - piezas.c & piezas.h
  - armas.c & armas.h
  - README.txt / README.md
  - **makefile**
- La tarea debe ser compilada utilizando el compilador **GCC** bajo el estándar **C11** (-std=c11)

- Se pueden añadir nuevos atributos a las estructuras, pero los nombres y firmas de las funciones obligatorias son inmutables.
- Se puede crear nuevas funciones, estructuras y/o definiciones que estime conveniente.
- La falta de declaración de los structs y funciones descritos en este documento en sus respectivos archivos de encabezado (.h) resultará en un 0 como nota final de la tarea.
- Es de **uso obligatorio la estructura Tablero y sus atributos para la entrega mínima**, así como la implementación de *Armas* a través de punteros a función como se describe en 3.2.4. de no usarse significará un 0 en la nota final de la tarea.
- Se permite únicamente el uso de la librería estándar de C. No está permitido incluir librerías externas, no estándar ni específicas del sistema operativo.
- Debe estar presente el archivo **Makefile** para que se efectúe la revisión, este debe compilar TODOS los archivos mediante compilación separada.
- Se utilizará Valgrind para detectar las fugas de memoria.
- Las funciones deben ir comentadas, explicando clara y brevemente lo que realiza, los parámetros que recibe y los que devuelve (en caso de que devuelva algo).

► **El Uso de IA está prohibido**. Cualquier sospecha de uso de herramientas de inteligencia artificial podrá conllevar descuentos en la evaluación o citación a defensa. En caso de comprobarse el uso de inteligencia artificial, la entrega será evaluada con nota 0 y notificada al profesor.)

- Si no existe orden en el código habrá descuento. **Se harán descuentos por funcion no comentada**
- La entrega debe realizarse en un archivo comprimido en tar.gz y debe llevar el nombre: Tarea2LP_RolAlumno.tar.gz.
  Ej.: Tarea2LP_202573000-k.tar.gz.
- El archivo README.txt debe contener el nombre y rol del alumno e instrucciones detalladas para la correcta utilización de su programa.
- La entrega será vía aula y el plazo máximo de entrega es hasta el día **16 de Abril, 23:59 horas, vía aula.**
- Por cada día de atraso se descontarán 20 puntos (10 puntos dentro de la primera hora)
- Las copias y Uso de código generado por IA serán evaluadas con nota 0 y se informarán a las respectivas autoridades.
- Solo se pueden realizar consultas respecto a la tarea hasta 2 días antes de la entrega.
- Las consultas del enunciado se debe realizarse mediante el foro de la tarea disponible en AULA.

6. Calificación

6.1. Entrega

Para la calificación de su tarea, debe realizar una entrega con requerimientos mínimos que otorgarán 30 pts base, luego se entregará puntaje dependiendo de los otros requerimientos que llegue a cumplir.

6.1.1. Entrega Mínima (total 30 pts)

Para obtener los 30 puntos base, el programa debe compilar y cumplir con los siguientes tres requerimientos fundamentales. De obtener estos puntos, se procederá a la revisión del resto de la tarea.

- **Gestión inicial de memoria (10 pts):** Inicializa el tablero del Nivel 1 (12x12), asignando espacio en la memoria *heap* para la matriz `void*** celdas` dentro del struct `Tablero`. *Notas adicionales:* El tablero debe inicializarse y liberarse obligatoriamente a través de `malloc` y `free`. El juego debe poder cerrarse sin presentar *leaks* de memoria en esta estructura básica.
- **Movimiento del Jugador y un Enemigo (10 pts):** El Rey puede moverse dentro de los límites del tablero usando las teclas designadas (WASD/QEZC). Además, al menos un tipo de pieza enemiga (ej. el Peón) logra acercarse al Rey turno a turno sin comportamientos inesperados (como salirse del tablero o el cierre inesperado del programa).
- **Punteros a Función y Disparo Básico (10 pts):** El arreglo de punteros a función `FuncArma` está correctamente implementado en la estructura `Armas`. El jugador es capaz de equipar al menos el arma 1 (Escopeta), elegir una dirección y disparar, consumiendo el turno.

De obtener los 30 puntos de la entrega mínima, se procederá a la revisión de las siguientes secciones de la tarea.

6.1.2. Asignación de puntajes general (total 70 pts)

Si el estudiante cumple con la entrega mínima, se evaluará el siguiente desglose:

1. **Arsenal y Punteros a Función (25 pts)**
   - **Escopeta:** Lógica de área cónica correcta (2 de daño al primero, 1 a los de atrás) y recarga de munición **exclusivamente** al caminar. **(3 pts)** (resto de mecánicas de la escopeta evaluadas en EM)
   - **Sniper:** Implementación correcta de la línea recta infinita ortogonal y diagonal con impacto único. **(5 pts)**
   - **Granada:** Implementación correcta del daño en área 3×3 (Granada) en diferentes direcciones y sin errores. **(5 pts)**
   - **Arma especial:** correcto funcionamiento (y originalidad/documentación) del arma o efecto especial diseñado por el estudiante. **(12 pts)**

   Todas las armas deben ser implementadas a través de punteros a función.

2. **Comportamiento y Pathfinding de Enemigos (20 pts)**

- **Peón y Caballo:** El Peón ataca ortogonal/diagonal correctamente a corta distancia. El Caballo salta en 'L' minimizando la distancia hacia el Rey, permitiendo saltar sobre otras piezas. **(6 pts)**
- **Alfil y Torre:** Movimientos diagonales y ortogonales respetados. La limitación de la Torre (se mueve 1 vez cada 2 turnos enemigos) funciona correctamente. **(6 pts)**
- **La Reina (Jefe Final):** Combina los movimientos ortogonales y diagonales, logrando un acecho implacable en el Nivel 3. **(4 pts)**
- **Reglas de colisión:** Solo existe una pieza por celda en todo momento. Los enemigos no se superponen al moverse. **(4 pts)**

3. **Flujo, Niveles y Memoria (12 pts)**
   - **Transición de Niveles:** El juego avanza correctamente del Nivel 1 al 2, y del 2 al 3 al limpiar la sala. La memoria del tablero anterior se libera por completo (`free`) y se aloja el nuevo tamaño sin fugas de memoria. **(4 pts)**
   - **Aparición (Spawn):** Las piezas aparecen en la cantidad correcta según el nivel. El Rey nace en la fila inferior y los enemigos en las filas superiores (aleatoriamente). **(4 pts)**
   - **Condiciones de Término:** El juego detecta la victoria al limpiar el Nivel 3, y detecta la derrota crítica (Jaque / pieza enemiga entra a la casilla del Rey). **(4 pts)**
4. **Interfaz, HUD y Turnos (13 pts)**
   - **Renderizado y HUD:** El tablero se imprime claramente en cada turno. El HUD muestra de forma precisa la munición, el nivel actual, las piezas restantes y el HP de los enemigos atacados. **(5 pts)**
   - **Ciclo de turnos:** El juego alterna correctamente entre el Rey y los enemigos. **(4 pts)**
   - **Validación de Entradas:** Si el jugador intenta moverse fuera del tablero, chocar contra una pared o disparar un arma sin munición, el juego avisa del error y **no consume el turno**. **(4 pts)**

6.2. Descuentos

- Falta de declaraciones en archivos .h (-100 pts)
- falta de uso de tablero y/o punteros a función (armas) (-100 pts)
- Representación gráfica del tablero inconsistente o confusa (-10 a 25 puntos según gravedad)
- Porcentaje de leak de memoria (bytes asignados) ((1 - 5) % -3 pts, (6 - 50) % -15 pts, (51 - 90) % -30 pts, (91 - 100) % - 40 pts))
- Falta de comentarios sobre cada función (–5 pts c/u, máx. –20 pts).
- Uso de librerías **no estándar** de C. (-100 pts)
- Falta de orden (Max -20 pts)
- Código no compila (-100 pts)
- Compilación Unificada (-5 pts)

- Warnings (-5 pts c/u)
- Falta de Makefile (-100 pts)
- Falta de README (–20 pts)
- Compilador o versión C errónea (-15 pts)
- Falta de información obligatoria en el README (nombre, rol, instrucciones) (–5 pts c/u)
- Falta de orden o mala indentación en el código (–5 a –20 pts según gravedad)
- Mal nombre en los archivos entregados (–5 pts c/u; –10 pts si es el `.tar.gz`)
- Uso de código y/o texto generado por IA implicará un 0 y se le informará a las respectivas autoridades.
- Entrega tardía (–10 pts si es dentro de la primera hora; –20 pts por cada día o fracción de retraso)
- En caso de existir nota negativa, esta será remplazada por un 0.